Design and Develop SQL Solutions Like a Pro

Microsoft Certified: SQL AI Developer Associate

Exam: **DP-800** I Role: **Developer** Level: **Intermediate**

Core Platforms

- Microsoft SQL Server
- Azure SQL
- SQL databases in Microsoft Fabric

Key Skills

- T-SQL development
- Designing relational and semi-structured data models
- CI/CD with GitHub
- AI fundamentals: embeddings, vectors, models
- AI-assisted development tools

Role

- Build secure, scalable database solutions
- Integrate AI into database-driven applications
- Optimize, deploy, and govern SQL workloads
- Work with App developers, DBAs, architects, AI and DevSecOps engineers, Security and compliance teams

Table of Contents

Module 1

Design & Implement Database Objects

Tables, indexes, constraints, JSON columns, partitioning, and specialized table types across SQL Server, Azure SQL, and Microsoft Fabric.

Platform Choices

SQL Server-Based Platforms

SQL Server On-Prem
Full control, Windows/Linux, self-managed

Docker / Kubernetes
Containers, dev/test parity, quick spin-up

Azure SQL DB / MI
Managed PaaS, built-in HA, serverless tiers

Fabric SQL Database
Unified analytics, OneLake, AI-ready Copilot

Table Design

Data Type Best Practices

- Choose appropriate data types and sizes to optimize storage and performance
- Normalize to third normal form and denormalize selectively for read-heavy workloads
- Every table needs a primary key that is narrow and immutable

Specialized Table Types

In-Memory OLTP	RAM-resident, lock-free MVCC, up to 30x faster OLTP. Natively compiled SPs.
Temporal	System-versioned history tracking. FOR SYSTEM_TIME AS OF queries.
External	Query remote data via PolyBase — Blob, ADLS, S3, Oracle. No data movement.
Ledger	Tamper-evident, cryptographic verification. Updatable or append-only.
Graph	NODE + EDGE tables with MATCH clause. Social, fraud, recommendation use cases.

Tip: Match the table type to your workload — temporal for audit, ledger for compliance, graph for relationships, in-memory for high-throughput.

Optimize with Indexes

Clustered

1 per table

Defines the physical sort order of table data

Nonclustered

Up to 999

Separate lookup structure for fast data retrieval

Columnstore

Analytics

Column-based storage for analytics and data warehouse workloads

Filtered

Subset

Indexes a subset of rows for targeted query optimization

Full-Text

Linguistic

Enables natural language search across text content

JSON Index

SQL 2025

Native indexing for JSON document paths in SQL Server 2025

Vector Index

SQL 2025

A vector index is a specialized index designed to optimize similarity searches on vector data, commonly used in applications like machine learning, recommendation systems, and AI. It enables efficient nearest neighbor searches by organizing vectors based on their relative distances

Index columns used in WHERE, JOIN, and ORDER BY clauses for best performance

Constraints & JSON Support

Enforce Data Integrity

PRIMARY KEY
Unique row ID, clustered index default, no NULLs

FOREIGN KEY
Referential integrity, CASCADE/SET NULL/NO ACTION

UNIQUE
Distinct values, allows one NULL, creates NC index

CHECK
Boolean validation: salary >= 15000 AND salary <= 100000

DEFAULT
Auto-assign on INSERT: GETDATE(), NEWID(), literals

JSON Columns & Indexes

- Native JSON data type in SQL Server 2025 — up to 2 GB per row
- JSON_VALUE, JSON_QUERY, JSON_MODIFY, OPENJSON for parse/transform
- JSON_OBJECT / JSON_ARRAY to construct JSON from SQL
- Computed column + standard index for specific $.paths (all versions)
- CREATE JSON INDEX: covers all paths without computed columns (2025+)
- Bridges relational + document models in a single platform
- ISJSON() validates JSON before storage; OPENJSON shreds to rows

Sequences & Partitioning

SEQUENCE vs. IDENTITY

Feature	SEQUENCE	IDENTITY
Scope	Database-level, shared across tables	Column-level, one table only
Pre-generate	NEXT VALUE FOR before INSERT	Only on INSERT
Cycling	Configurable CYCLE / NO CYCLE	Not supported
Cross-server	Works on linked servers	SET IDENTITY_INSERT issues

Partitioning Strategy

1. **Partition Function**
2. **Partition Scheme**
3. **Create Table**
4. **Aligned Indexes**

- Partition elimination — queries scan only relevant partitions
- Instant archival with ALTER TABLE SWITCH
- Rebuild indexes per partition — minimize downtime
- Sliding window for automated data lifecycle management

Module 2

Implement Programmability Objects

Views, stored procedures, scalar functions, table-valued functions, and triggers for maintainable, secure, and efficient database solutions.

Views

Virtual tables that simplify access, enforce security, and abstract schema changes.

Standard Views

- Encapsulate complex JOINs into reusable queries
- Grant SELECT on view without exposing base tables
- WITH CHECK OPTION prevents inserts violating the view's WHERE

Indexed Views (Materialized)

- WITH SCHEMABINDING + unique clustered index
- Physically stores result set, Fast for aggregation-heavy reads
- Auto-maintained on DML; consider write overhead

Partitioned Views

- UNION ALL across member tables with CHECK constraints
- Distributed partitioned views span multiple servers
- Query optimizer routes to relevant member tables only

Best Practices

Use SCHEMABINDING to prevent accidental base-table changes • Avoid SELECT * • Layer views for complex security rules • Don't nest views beyond 2 levels for readability

Stored Procedures

Server-side compiled programs that encapsulate business logic, reduce network round-trips, and enforce security.

Core Capabilities

- INPUT and OUTPUT parameters for flexible data exchange
- RETURN values for status codes and execution flow
- Temp tables and table variables for intermediate results
- Dynamic SQL with sp_executesql for runtime query building
- TRY...CATCH + transactions for atomic operations
- EXECUTE AS for security context switching
- SET NOCOUNT ON to suppress row-count messages

Design Patterns

- CRUD operations: encapsulate INSERT/UPDATE/DELETE logic
- ETL pipelines: orchestrate data movement and transformation
- Report generation: pre-aggregate data for dash
- Natively compiled SPs: compile to machine code for in-memory tables
- Grant EXECUTE without direct table access security layer
- Plan caching: compiled once, reused but watch for parameter sniffing
- Use OPTION (RECOMPILE) or OPTIMIZE FOR when plans skew

User-Defined Functions

Scalar Functions

Return a single value (INT, VARCHAR, etc.)

- Callable in SELECT, WHERE, CHECK constraints
- Deterministic = indexable computed columns
- Caution: row-by-row execution can hurt large datasets
- Use WITH SCHEMABINDING for optimization

Inline Table-Valued (iTVF)

Single SELECT returning a table — best performance

- Inlined by optimizer like a parameterized view
- Supports JOIN, WHERE, aggregation in the body
- Preferred over multi-statement TVFs for read queries
- Can reference in CROSS APPLY / OUTER APPLY

Multi-Statement TVF

Multiple statements, explicit RETURNS TABLE variable

- Populate @table with INSERT...SELECT loops
- Not inlined — often estimated at 1 or 100 rows
- Use for complex logic that can't be a single query
- SQL Server 2019+ interleaved execution helps cardinality

Triggers

Automatically respond to data modifications or database events — use sparingly for audit, validation, and enforcement.

DML Triggers

AFTER / INSTEAD OF

- Fire on INSERT, UPDATE, DELETE on tables/views
- AFTER: runs after the DML statement completes
- INSTEAD OF: replaces the DML — enables updatable views
- Access inserted/deleted pseudo-tables for old/new values

DDL Triggers

Database / Server

- Respond to CREATE, ALTER, DROP events
- Database-scoped or server-scoped
- EVENTDATA() returns XML with event details
- Audit schema changes or prevent unauthorized DDL

Logon Triggers

Server-Level

- Fire on LOGON events after authentication
- Control session creation based on time/IP/count
- Limit concurrent sessions per login
- Use carefully — can lock out all users if broken

Best Practices

Keep trigger logic minimal • Avoid nested triggers • Never rely on trigger firing order • Consider temporal tables as an alternative for audit • Test with multi-row operations

Module 3

Write Advanced T-SQL Code

CTEs, window functions, JSON processing, regular expressions, fuzzy matching, graph queries, and structured error handling.

CTEs & Correlated Subqueries

Common Table Expressions

- WITH clause defines named, temporary result sets
- Scope: single SELECT, INSERT, UPDATE, or DELETE statement
- Recursive CTEs for hierarchical data: org charts, BOMs, tree traversal
- Anchor member + recursive member joined by UNION ALL
- MAXRECURSION hint controls depth (default 100, max 32767)
- Multiple CTEs in a single WITH — separate with commas
- Improves readability vs deeply nested subqueries
- Not persisted — recalculated each reference (use temp tables if needed twice)

Correlated Subqueries

- Inner query references columns from the outer query
- Executes once per outer row — watch for $O(n^2)$ performance
- EXISTS / NOT EXISTS for semi-join patterns
- Row-by-row comparisons: find max per group, running differences
- Can often be rewritten as JOINs or window functions for speed
- Useful in UPDATE/DELETE with correlated conditions
- Derived tables: subquery in FROM clause with an alias
- CROSS APPLY / OUTER APPLY for table-valued expressions

Window Functions

Perform calculations across rows related to the current row, without collapsing result sets like GROUP BY.

Ranking

ROW_NUMBER
RANK
DENSE_RANK
NTILE

Assign positions within partitions. ROW_NUMBER is always unique; RANK allows ties.

Offset

LAG / LEAD
FIRST_VALUE
LAST_VALUE

Access previous/next rows or partition boundaries without self-joins.

Aggregate

SUM / AVG
COUNT / MIN
MAX

Running totals, moving averages over ROWS BETWEEN or RANGE frames.

Distribution

PERCENT_RANK
CUME_DIST
PERCENTILE_CONT
PERCENTILE_DISC

Statistical distribution and percentile calculations within groups.

Syntax: function() OVER (PARTITION BY col ORDER BY col ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

JSON & XML Processing in T-SQL

JSON Functions

- FOR JSON PATH / AUTO — convert result sets to JSON
- OPENJSON — shred JSON arrays/objects into rows with schema
- JSON_VALUE — extract scalar from $.path expression
- JSON_QUERY — extract object or array sub-document
- JSON_MODIFY — update values in-place without full rewrite
- JSON_OBJECT / JSON_ARRAY — construct JSON (SQL 2022+)
- ISJSON — validate JSON before storage or processing
- JSON_CONTAINS — check if value exists (SQL 2025)

XML Functions

- FOR XML PATH / RAW / AUTO — convert to XML output
- .query() — XQuery expressions on XML columns
- .value() — extract scalar from XPath expression
- .modify() — insert, delete, or replace XML nodes
- .nodes() — shred XML into relational rows
- OPENXML + sp_xml_preparedocument for legacy parsing
- XML indexes: primary + secondary (PATH, VALUE, PROPERTY)
- XML Schema Collections for strong typing and validation

SQL Server 2025: Regex & Fuzzy Matching

Regular Expressions

- REGEXP_LIKE — filter rows matching a pattern (replaces complex LIKE chains)
- REGEXP_REPLACE — pattern-based string replacement with capture groups
- REGEXP_SUBSTR — extract substrings matching a pattern
- REGEXP_COUNT — count occurrences of a pattern in text
- SARGable: can use indexes for efficient execution
- ICU regex engine — full Unicode support, POSIX classes
- Use cases: email validation, phone parsing, data cleansing, log analysis

Fuzzy String Matching

- EDIT_DISTANCE — Levenshtein distance between two strings
- EDIT_DISTANCE_SIMILARITY — percentage similarity (0–100)
- SOUNDEX — phonetic encoding for English names
- DIFFERENCE — compare SOUNDEX codes (0–4 scale, 4 = exact)
- Combine with thresholds: WHERE EDIT_DISTANCE_SIMILARITY > 80
- Use cases: deduplication, address matching, customer name resolution
- Pair with regex: clean first (regex) → match (fuzzy) → deduplicate

Graph Queries & Error Handling

Graph Queries (MATCH)

- NODE tables store entities; EDGE tables store relationships
- MATCH clause: WHERE MATCH(Person-(Follows)->Person)
- SHORTEST_PATH for finding shortest route between nodes
- Variable-length traversal with + quantifier
- CONNECTION constraint restricts which nodes an edge can connect
- Combine graph + relational queries in single T-SQL statements
- Use cases: social networks, fraud rings, org hierarchies, recommendations

TRY...CATCH Error Handling

- TRY block wraps statements; CATCH block handles errors
- ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_SEVERITY(), ERROR_LINE()
- THROW — re-raise or raise custom errors (replaces RAISERROR)
- XACT_ABORT ON: auto-rollback on any error — simplifies cleanup
- Nest TRY...CATCH with transactions for atomic multi-step operations
- @@TRANCOUNT checks active transaction depth before COMMIT/ROLLBACK
- Log errors to a table in CATCH before re-throwing

Module 4

AI-Assisted Database Development

Accelerate SQL authoring with GitHub Copilot, Fabric Copilot, and Model Context Protocol (MCP) for agent-driven database operations.

GitHub Copilot for SQL

AI-powered assistant inside SSMS 22 and VS Code — schema-aware, version-specific T-SQL generation.

Natural Language to SQL

Describe what you need in plain English. Copilot generates schema-aware T-SQL with correct table/column references and JOINs.

Code Completions

Real-time "ghost text" suggestions as you type. Tab to accept, keep typing to ignore. Reduces syntax errors and boilerplate.

Explain & Optimize

Select any query and ask Copilot to explain what it does or suggest optimizations. Great for debugging legacy code.

Schema Awareness

Reads your active database connection — knows SQL version, tables, columns, keys, and relationships for contextual suggestions.

Requires SSMS 22+ or VS Code MSSQL extension • GitHub Copilot subscription (Free tier: 50 requests) • Respects DB permissions

Fabric Copilot for SQL

AI assistant in the Fabric SQL database workload — natural language queries, code completion, and quick actions.

Chat Pane	Ask natural language questions about your database. Get T-SQL queries or documentation answers directly in the Fabric portal.
Code Completion	Start typing in the SQL query editor — Copilot suggests completions for table names, functions, and full queries. Tab to accept.
Quick Actions: Fix	Highlight a failing query and click Fix. Copilot identifies errors, corrects syntax, and adds comments explaining changes.
Quick Actions: Explain	Select any SQL block and click Explain. Copilot provides natural language explanation of query logic and schema usage.

Built on same AI tools as SSMS Copilot • Consistent experience across Fabric portal, SSMS 22, and VS Code

Model Context Protocol (MCP)

Open protocol standardizing how AI agents safely interact with databases — a "USB-C port" for AI applications.

AI Client

VS Code, Claude,
ChatGPT, Copilot

→

MCP Server

SQL MCP Server
(Data API Builder)

→

SQL Server

On-prem, Azure SQL,
Fabric, Docker

SQL MCP Server Tools

- ListTables — discover available tables and schemas
- ReadData — query rows with filters and pagination
- CreateItem — insert new rows with typed values
- UpdateItem — modify existing rows by key
- DeleteItem — remove rows with key-based targeting
- ExecuteQuery — run custom T-SQL with RBAC controls

Security & Architecture

- Built on Data API Builder — proven RBAC and telemetry
- Role-based access control for each entity and operation
- Deploy locally (stdio transport) or to Azure Container Apps
- Supports SQL Server, Azure SQL, Fabric SQL databases
- Entity descriptions enrich AI context for better queries
- No data leaves your environment — runs alongside your database

AI Workflow Integration

Combining tools: Copilot for authoring, MCP for live data access, Fabric for unified analytics.

Author

Use GitHub Copilot in SSMS/ VS Code to write and optimize T-SQL. Schema-aware suggestions accelerate development.

Connect

SQL MCP Server bridges AI agents to your database. Agents discover schemas and run queries through standard protocol.

Analyze

Fabric Copilot enables natural language exploration in the portal. Quick actions fix errors and explain complex logic.

Govern

Copilot instruction files (.github/ copilot-instructions.md) enforce coding standards. RBAC in MCP controls data access.

Best practice: Always review AI-generated SQL before production • Use EXPLAIN to validate query plans • Start with read-only access for AI agents

When to Use What — Decision Guide

High-throughput OLTP with latch contention	**In-memory OLTP tables + natively compiled SPs**
Audit trail / data forensics / compliance	**Ledger tables (updatable or append-only)**
Query external storage without import	**External tables with PolyBase / REST APIs**
Complex relationships (social, fraud)	**Graph tables + MATCH + SHORTEST_PATH**
Reusable read-only access layers	**Views (standard or indexed for aggregation)**
Complex pattern matching / text parsing	**Regex functions (SQL 2025) + full-text index**
AI-powered query authoring	**GitHub Copilot in SSMS 22 / VS Code**
AI agent database operations	**SQL MCP Server + Data API Builder + RBAC**

Key Takeaways

- SQL Server runs everywhere — on-premises, Docker, Azure PaaS, and Fabric
- Match table types to workloads: temporal for audit, ledger for compliance, graph for relationships
- Choose the right index: clustered for OLTP, columnstore for analytics, JSON index for 2025+
- Encapsulate logic in SPs and functions — use views for secure, reusable read access
- Window functions and CTEs replace complex self-joins and correlated subqueries
- SQL Server 2025 adds native regex, fuzzy matching, and JSON type improvements
- GitHub Copilot + Fabric Copilot accelerate SQL authoring with schema-aware AI
- MCP connects AI agents to databases safely with RBAC and standard protocols

References

Bookmark these — they cover the official exam page and the full video series.

01

Official DP-800 certification page

https://learn.microsoft.com/en-us/credentials/certifications/developing-ai-enabled-database-solutions/?practice-assessment-type=certification

02

Community series — Microsoft Reactor (Data Days: The SQL AI Series)

https://developer.microsoft.com/en-us/reactor/series/S-1683/?wt.mc_id=datadayslive_S-1683_email_reactor

Thank you, crew!

You've got the skills to design robust database objects and write advanced, maintainable T-SQL — now go build something great.